

前言

01.学习概述

1.1 学习目标

1.2 前置知识

02.核心概念

2.1 是什么？

2.2 解决什么问题？

2.3 基本特性

03.整体统计梳理

04. 各功能域详细分析

4.1【功能域1：视频播放核心】● 极高复杂度

4.2【功能域2：录像回放与时间轴】● 高复杂度

4.3【功能域3：设备管理与切换】● 高复杂度

4.4【功能域4：语音对讲】● 极高复杂度

4.5【功能域5：云台控制PTZ】● 中高复杂度

4.6【功能域6：截图录像与水印】● 中高复杂度

4.7【功能域7：AI事件管理】● 中高复杂度

4.8【功能域8：AI检测配置】● 高复杂度

4.9【功能域9：连续浏览多画面】● 高复杂度

4.10【功能域10：图片编辑与涂鸦】● 中高复杂度

05.当前已做

06.迁移风险总结

其他介绍

01.关于我的博客

前言

学习要符合如下的标准化链条：了解概念->探究原理->深入思考->总结提炼->底层实现->延伸应用"

01.学习概述

• 学习主题：

• 知识类型：

☐ ☒ Android/

☐ ☒ 01.基础组件与机制

☐ ☒ 四大组件

☐ ☒ IPC机制

☐ ☒ 消息机制

☐ ☒ 事件分发机制

☐ ☒ View与渲染体系（含Window、复杂控件、动画）

☐ ☒ 存储与数据安全（SharedPreferences/DataStore/Room/Scoped Storage）

☐ ☒ 02. 架构与工程化

☐ ☒ 架构模式（MVC/MVP/MVVM/MVI）

☐ ☒ 依赖注入（Koin/Hilt/Dagger）

☐ ☒ 路由与模块化（ARouter、Navigation）

- ☐ ☒ Gradle与构建优化
- ☐ ☒ 插件化与动态化
- ☐ ☒ 插桩与监控框架
- ☐ ☒ 03.性能优化与故障诊断
 - ☐ ☒ ANR分析与优化
 - ☐ ☒ 启动耗时优化
 - ☐ ☒ 内存泄漏监控
 - ☐ ☒ 监控与诊断工具
- ☐ ☒ 04.Jetpack与生态框架
 - ☐ ☒ Room
 - ☐ ☒ Paging
 - ☐ ☒ WorkManager
 - ☐ ☒ Compose
- ☐ ☒ 05.Framework与系统机制
 - ☐ ☒ ActivityManagerService (含ANR触发机制)
 - ☐ ☒ Binder机制
- ☐ ☒ 音视频开发/
 - ☐ ☒ 01.基础知识
 - ☐ ☒ 02.OpenGL渲染视频
 - ☐ ☒ 03.FFmpeg音视频解码
- ☐ ☒ Java/
 - ☐ ☒ 01.基础知识
 - ☐ ☒ 02.集合框架
 - ☐ ☒ 03.异常处理
 - ☐ ☒ 04.多线程与并发
 - ☐ ☒ 06.JVM
- ☐ ☒ Kotlin/
 - ☐ ☒ 01.基础语法
 - ☐ ☒ 02.高阶扩展
 - ☐ ☒ 03.协程和流
- ☐ ☒ Flutter/
 - ☐ ☒ 01.基础知识
 - ☐ ☒ Dart 语言基础
 - ☐ ☒ Widget 基础与生命周期
 - ☐ ☒ Flutter 基础组件
 - ☐ ☒ 布局与约束
 - ☐ ☒ 绘制与渲染体系
 - ☐ ☒ 状态管理
 - ☐ ☒ 事件处理与手势系统

- ☐ ☒ 原生通信
- ☐ ☒ 02.路由与导航
- ☐ ☒ 03.性能优化与故障诊断
- ☐ ☒ 04.异步编程
- ☐ ☒ 05.项目经验与案例沉淀
- ☐ ☒ 自我管理/
 - ☐ ☒ 01.内观
- ☐ ☒ 项目经验/
 - ☐ ☒ 01.启动逻辑
 - ☐ ☒ 02.云值守
 - ☐ ☒ 03.智控平台
 - ☐ ☒ 04.视频巡店
- 学习来源:
- 重要程度: ★ ★ ★ ★ ★
- 学习日期: 2025.
- 记录人: @panruiqi

1.1 学习目标

- 了解概念->探究原理->深入思考->总结提炼->底层实现->延伸应用"

1.2 前置知识



02.核心概念

2.1 是什么?

2.2 解决什么问题?

2.3 基本特性

03.整体统计梳理

- 整体梳理具体如下:

- 视频巡店业务线 (lib_video + 5个依赖模块)
 - |
 - └─ 【核心模块 lib_video】
 - | └─ Activity层(6个): VideoActivity、ReplayActivity、VideoSplitActivity等
 - | └─ Fragment层(15个): VideoPlayFragment、VideoDownloadFragment等

- | └ widget层(35个): VideoPlayView、IjkPlayView、DragPointView等
- | └ Presenter层(10个): MVP业务逻辑层
- | └ Adapter层(34个): RecyclerView适配器
- | └ Manager层(5个): PlayManager、StorageCheckerManager等
- | └ Utils层(20个): VideoViewUtils、BuildUrl等
- | └ ViewModel层(4个): LiveData状态管理
- |
- | └ 【图片编辑 lib_edit】(8000行)
- | └ 【媒体浏览 lib_media_viewer】(3000行)
- | └ 【巡店公共 lib_patrol_shop】(12000行)
- | └ 【第三方SDK】
 - | └ IjkPlayer(FFmpeg)
 - | └ 萤石/乐橙/悠络客SDK
 - | └ VoiceIntercomManager
 - | └ timerulerslib

- 不可压缩的技术风险

风险项	影响范围	迁移工作量	缓解方案
4种播放器SDK不可用	12,000行	4-5周	统一降级RedPlayer (HTTP-FLV)
语音对讲库重写	3,000行	2-3周	AVRecorder+G.711编解码重写
时间轴组件重写 (很多是Android三方库组件)	2,000行	2-3周	ArkTS自定义组件
VideoActivity重构	6,089行	3-4周	拆分为6个Feature模块
FFmpeg重新编译	Native层	1-2周	RedPlayer编译脚本

- 鸿蒙上的三方SDK较少，比如：视频播放模块，redplayer不支持rtsp，需要使用鸿蒙工具链重编译libffmpeg，这个很耗时间，因为鸿蒙工具链处理这个的笔记较少，需要自己探索。
- 代码需要重构，之前的技术债（迫于需求交付压力导致的代码集成）：VideoActivity。6089行。VideoPlayFragment5390行。需要拆分重构架构，修改为MVVM架构。
- 需要的基础组件：播放器，libffmpeg,萤石/乐橙/悠络客SDK（尚不清楚他们是否适配鸿蒙）

- 整体工期评估（含架构重构）

场景	功能范围	工期	人力	说明
最低可用P0	播放+回放+设备（不含语音/AI/编辑）	12-14周	2人	不重构，功能阉割
合理迁移P0+P1	+语音+云台+截图（不含AI/编辑）	18-22周	2-3人	部分重构
完整对等	全功能+架构完全重构	26-32周	3-4人	推荐方案

- 完整统计

- 总代码量：~150,000行Java/Kotlin
- 总类数：275+个类
- 最低合理工期：18-22周（4.5-5.5个月）
- 推荐完整工期：26-32周（6.5-8个月）

04. 各功能域详细分析

4.1 【功能域1：视频播放核心】🔴 极高复杂度

模块清单

- 如下

类名	类别	代码量	复杂度	核心依赖	使用的Utils	第三方库	迁移风险	核心流程
VideoActivity	Activity	6,089行	极高	所有Fragment+ViewModel	VideoViewUtils, VideoPlayUtils, BuildUrl, SDKConstants	EventBus, ARouter, VoiceIntercomManager	🔴 极高：需拆分为6个子模块	☑ 是
VideoPlayFragment	Fragment	5,390行	极高	4种播放器SDK	VideoViewUtils, BuildUrl, VideoPlayUtils, MediaPlayerTool	IjkPlayer, EZPlayer, LCPlayer, UluPlayer	🔴 极高：播放器适配全部重写	☑ 是
VideoPlayView	Widget	1,889行	极高	4种播放器SDK	VideoViewUtils, SDKConstants	直接调用4种播放器	🔴 极高：SurfaceView→XComponent	☑ 是
IjkPlayView	Widget(java)	1,893行	高	IjkPlayer	Settings	IjkMediaPlayer(JNI)	🔴 极高：FFmpeg需重新编译	☑ 是
VideoPlayPresenter	Presenter	~800行	中	VideoApi	VideoPlayUtils	-	🟡 中：MVP→ViewModel	☑ 是
VideoPlayUtils	工具类	~600行	中高	4种SDK	BuildUrl, VideoViewUtils	4种播放器API	🟡 中：URL构建逻辑	☑ 是
VideoServicePlayUtils	工具类	~500行	中	4种SDK	BuildUrl	4种播放器	🟡 中	✗ 否
MediaPlayerTool	工具类	~400行	中	IjkPlayer	-	IjkMediaPlayer	🟡 中	☑ 是
VideoPlayService	Service	~600行	中	4种SDK	VideoServicePlayUtils	4种播放器	🟡 中：前台Service	✗ 否
PlayManager	Manager	~300行	低	VideoPlayView	-	-	🟢 低	☑ 是
BuildUrl	工具类	~800行	中高	无	-	-	🟡 中：URL规则需保持	☑ 是
VideoViewUtils	工具类	~700行	中	4种SDK	-	4种播放器错误码	🟡 中：错误码映射	☑ 是

- 迁移风险点
 - 播放器SDK不可用风险 🔴
 - 萤石/乐橙/悠络客是否支持鸿蒙？若无SDK，需降级为HTTP-FLV统一用RedPlayer
 - IjkPlayer需替换为RedPlayer（FFmpeg重新编译为鸿蒙.so）
 - 影响范围：5个核心类，~12,000行代码需重写
 - VideoActivity上帝类 🔴
 - 6089行单文件，包含260+成员变量
 - 必须重构：拆分为6个Feature模块（播放、设备、回放、云台、事件、AI）
 - 架构改造：MVP → MVVM + Observed状态管理
 - SurfaceView → XComponent 🔴
 - Android的SurfaceView在鸿蒙需改为XComponent
 - 涉及播放器渲染、生命周期、触摸事件全部重写
- 依赖关系
 - 对外依赖：VideoApi、OkHttp、EventBus、ARouter、VoiceIntercomManager
 - 对内依赖：所有Utils、所有Fragment、ViewModel
 - 第三方库：IjkPlayer(FFmpeg)、萤石SDK、乐橙SDK、悠络客SDK
- 工期评估
 - 不重构：5-6周（2人）
 - 含重构：8-10周（2-3人）
 - 架构重构额外工期：3-4周

4.2 【功能域2：录像回放与时间轴】 ● 高复杂度

- 模块清单

○	类名	类别	代码量	复杂度	核心依赖	使用的Utils	第三方库	迁移风险	核心流程
	VideoDownloadFragment	Fragment	3,300行	高	EZOpen, LC, ULU	VideoUtils, DateUtils	EZOpen 下载SDK	● 高：时间轴组件需重写	☑ 是
	VideoDownloadPresenter	Presenter	~600行	中	VideoApi	VideoUtils	-	○ 中	☑ 是
	DownloadVideoService	Service	1,200行	中高	EZOpen, LC	VideoFileManager	EZOpen 下载SDK	● 高：下载SDK需适配	☑ 是
	DownloadVideoForAbnormalService	Service	~500行	中	同上	同上	同上	○ 中	✗ 否
	RulerView	第三方库	~2,000行	高	timerulerslib	-	第三方库	● 极高：需完全重写	☑ 是
	VideoUtils	工具类	~500行	中	无	-	-	● 低	☑ 是
	VideoCacheManager	工具类	~400行	中	数据库	-	-	○ 中：数据库API不同	☑ 是
	VideoFileManager	工具类	~600行	中	文件系统	-	-	○ 中：文件路径不同	☑ 是
	ReplayActivity	Activity	1,100行	中	IjkPlayer	VideoViewUtils	IjkPlayer	○ 中	☑ 是

- 迁移风险点

- 时间轴组件必须重写 ●

- RulerView是第三方Java库（~2000行），鸿蒙无法直接使用
 - 必须用ArkTS重写：24小时时间轴、录像段标记、滑动选择
 - 工作量：2-3周

- 录像下载SDK适配 ●

- 萤石/乐橙的录像下载API可能不支持鸿蒙
 - 需验证SDK是否提供鸿蒙版本，否则降级HTTP下载

- 断点续传实现 ●

- Android的DownloadManager在鸿蒙需改为Request API
 - 断点续传逻辑需重新实现

- 工期评估

- 不含时间轴：2-3周
 - 含时间轴重写：4-5周
 - 风险缓冲：+1周

4.3 【功能域3：设备管理与切换】 ● 高复杂度

- 模块清单

类名	类别	代码量	复杂度	核心依赖	迁移风险	核心流程
VideoActivity(设备管理部分)	Activity	~1,500行	高	VideoApi	● 高：逻辑混乱需拆分	☑ 是
VideosListDropDownView	Widget	~400行	中	RecyclerView	● 低	☑ 是
VideoViewModel	ViewModel	~300行	中	LiveData	● 中：LiveData→Observed	☑ 是
Device(model)	数据类	~150行	低	无	● 低	☑ 是
RvcVo(model)	数据类	~100行	低	无	● 低	☑ 是

- 迁移风险点
 - LiveData → Observed
 - Android的LiveData在鸿蒙需改为Observed + ObjectLink
 - 状态管理机制不同
 - 设备列表管理复杂 ●
 - 多设备切换、状态同步、能力查询
 - 横竖屏切换时的状态保持**
- 工期评估
 - 基础迁移：1-2周

4.4 【功能域4：语音对讲】 ● 极高复杂度

- 模块清单

类名	类别	代码量	复杂度	核心依赖	第三方库	迁移风险	核心流程
VoiceIntercomManager	第三方SDK	未知(外部库)	极高	AudioRecord	第三方库	● 极高：需完全重写	☑ 是
VideoActivity(对讲部分)	Activity	~600行	中	VoiceIntercomManager	-	● 中	☑ 是
AudioPopWindow	Widget	~200行	低	无	-	● 低	✗ 否

- 迁移风险点
 - 语音库必须完全重写 ●
 - VoiceIntercomManager依赖Android AudioRecord
 - 鸿蒙需改为AVRecorder + AudioCapturer
 - G.711编解码需重新实现
 - WebSocket音频流传输需重写
 - 工作量：2-3周
 - 权限申请机制不同 ●
 - 麦克风权限申请流程需适配
- 工期评估

- **语音库重写**: 2-3周
- **业务集成**: 1周
- **总计**: 3-4周

4.5 【功能域5：云台控制PTZ】 🌀 中高复杂度

- 模块清单

类名	类别	代码量	复杂度	迁移风险	核心流程
VideoActivity(PTZ部分)	Activity	~800行	中高	🌀 中：触摸事件重写	☑ 是
CircleControlView	Widget	~500行	中	🌀 中：Canvas绘制	☑ 是
PtzSpeedPopWindow	Widget	~200行	低	🌀 低	☑ 是
VideoPictureFragment(预置位)	Fragment	~850行	中	🌀 中	☑ 是

- 迁移风险点
 - **触摸事件API不同** 🌀
 - ACTION_DOWN/MOVE/UP → onTouch事件
 - 手势识别需重新适配
 - **Canvas绘制** 🌀
 - 云台摇杆的Canvas绘制逻辑需适配鸿蒙Canvas API
- 工期评估
 - **基础迁移**: 1周
 - **充分测试**: 1-2周

4.6 【功能域6：截图录像与水印】 🌀 中高复杂度

- 模块清单

类名	类别	代码量	复杂度	核心依赖	迁移风险	核心流程
VideoPlayFragment(截图录像)	Fragment	~1,000行	中高	OSS, 4种播放器	🌀 中高：截帧API不同	☑ 是
VideoPictureFragment	Fragment	~850行	中	数据库	🌀 中	☑ 是
WaterMarkUtils	工具类	~400行	中	Canvas	🌀 中：Canvas绘制	☑ 是
VideoRecordUtils	工具类	~300行	中	MediaRecorder	🌀 高：录像API不同	☑ 是

- 迁移风险点
 - 播放器截帧API差异
 - 4种播放器的截图API不同
 - RedPlayer的截图API需验证
 - OSS上传
 - 阿里云OSS SDK需验证鸿蒙兼容性
 - 本地录像
 - MediaRecorder → AVRecorder
 - 录像文件管理需重写
- 工期评估
 - 基础功能：1-2周
 - 含录像重写：2-3周

4.7 【功能域7：AI事件管理】 中高复杂度

- 模块清单

类名	类别	代码量	复杂度	迁移风险	核心流程
VideoEventFragment	Fragment	1,400行	中高	中高：时间轴组件	☑ 是
VideoEventPresenter	Presenter	~500行	中	中	☑ 是
VideoAddEventFragment	Fragment	~150行	低	低	✕ 否
ScaleTimeRulerView	第三方库	~800行	中	中：需重写	☑ 是
20+个事件相关Adapter	Adapter	~2,500行	中	低	✕ 否

- 工期评估：3-4周

4.8 【功能域8：AI检测配置】 高复杂度

- 模块清单

类名	类别	代码量	复杂度	迁移风险	核心流程
DetectionConfigActivity	Activity	~1,800行	高	● 高: Canvas+触摸	☑ 是
DragPointView	Widget	~400行	中	● 中: 触摸拖拽	☑ 是
RoundedSquareView	Widget	~300行	中	● 中: Canvas绘制	☑ 是
DragPointUtils	工具类	~300行	中	● 中: 坐标转换	☑ 是

- 迁移风险点
 - Canvas绘制+拖拽●
 - 检测区域矩形框绘制
 - 拖拽点触摸事件
 - 坐标转换计算
- 工期评估: 3-4周

4.9 【功能域9：连续浏览多画面】● 高复杂度

- 模块清单

类名	类别	代码量	复杂度	迁移风险	核心流程
VideoScanFragment	Fragment	1,500行	高	● 高: 10播放器池	☑ 是
VideoSplitActivity	Activity	1,200行	高	● 高: 4画面分屏	☑ 是
VideoSplitAllActivity	Activity	~900行	高	● 高: 9画面分屏	☑ 是
VideoScanAdapter	Adapter	~600行	中	● 中	☑ 是

- 迁移风险点
 - 多播放器并发管理●
 - 10个播放器池内存管理
 - RecyclerView复用机制（这是Android的，鸿蒙中的该考虑如何处理呢？）
 - 播放器生命周期同步
- 工期评估: 2-3周

4.10 【功能域10：图片编辑与涂鸦】● 中高复杂度

- 模块清单

- | 模块 | 代码量 | 复杂度 | 迁移风险 |
|------------------|---------|-----|--------------|
| lib_edit | ~8,000行 | 高 | ● 高：裁剪/滤镜/水印 |
| lib_media_viewer | ~3,000行 | 中 | ● 中：图片缩放预 |
| | | | |
- 工期评估：3-4周

05.当前已做

已完成工作

- **语音对讲库重构与迁移**：完成对讲库重写并迁移到鸿蒙平台，确保核心逻辑在新平台可用。（和小青蛙一起测试了宇树的对讲逻辑，没有问题）

本周在处理：

- 播放器接入：
 - 将原本以 HRA 包形式提供的 RedPlayer 转换为源码编译模式，同时解决组员在编译过程中遇到的失败问题。
排查 RedPlayer 不支持 RTSP 的问题，目前定位为 FFmpeg 支持不足，已提交 Issue：[REDPlayer #68](#)。
- 视频巡店部分架构重构
 - **UI组合式**：
 - 使用 Component 创建可复用 UI 组件（如 `DeviceSwitchBar`）。
 - 组件通过 `consume` 获取全局状态，只负责 UI 渲染，不承担业务逻辑。
 - **状态分散式**：
 - `VideoPatrolState` 作为全局状态聚合器。
 - 每个功能模块拥有独立状态：`DeviceState`、`VideoPlaybackState`、`VoiceIntercomState`。
 - 使用 `Observed + Track` 实现响应式，利用 `Provide/Consume` 实现状态共享。
 - **功能模块式**：
 - 按业务域划分目录，如 `feature_video_playback`、`feature_voice_intercom`。
 - 每个模块包含自己的 State、ViewModel、Manager，通过 DI 容器实现解耦。
 - 使用 Orchestrator 统一编排跨模块交互。
- 业务实现
 - **阶段一**：完成页面跳转及参数解析，DI 容器注册已完成。
 - **阶段二**：成功加载设备能力信息（RVC + NVR），确保 Orchestrator 初始化时数据就绪。
 - **阶段三（进行中）**：实现视频播放器调用逻辑及播放器状态变更，目前 UI 层尚未完成。

06.迁移风险总结

- 不可压缩的技术风险

○	风险项	影响范围	迁移工作量	缓解方案
	4种播放器SDK不可用	12,000行	4-5周	统一降级RedPlayer (HTTP-FLV)
	语音对讲库重写	3,000行	2-3周	AVRecorder+G.711编解码重写
	时间轴组件重写 (很多是Android三方库组件)	2,000行	2-3周	ArkTS自定义组件
	VideoActivity重构	6,089行	3-4周	拆分为6个Feature模块
	FFmpeg重新编译	Native层	1-2周	RedPlayer编译脚本

- 整体工期评估 (含架构重构)

○	场景	功能范围	工期	人力	说明
	最低可用P0	播放+回放+设备 (不含语音/AI/编辑)	12-14周	2人	不重构, 功能阉割
	合理迁移P0+P1	+语音+云台+截图 (不含AI/编辑)	18-22周	2-3人	部分重构
	完整对等	全功能+架构完全重构	26-32周	3-4人	推荐方案

- 关键结论
 - 实际代码规模: ~150,000行Java/Kotlin
 - 最低合理工期: 18-22周 (4.5-5.5个月)
 - 推荐完整工期: 26-32周 (6.5-8个月)
 - 1周交付可行性: 0%, 技术上完全不可能

其他介绍

01.关于我的博客

- csdn: http://my.csdn.net/qg_35829566
- 掘金: <https://juejin.im/user/499639464759898>
- github: <https://github.com/jjjjjjava>
- 邮箱: [\[934137388@qq.com\]](mailto:934137388@qq.com)

- ○ ■